56

用密码学保护信息

本章作者：**Peter Reiher（UCLA）**

56.1 引言

在前面几章中，我们讨论了如何明确安全目标、如何确定安全策略（security policy）、如何用身份认证（authentication）机制识别主体，以及如何用访问控制机制（access control mechanism）实施策略，规定哪些主体可以以哪些方式访问哪些计算机资源。虽然我们指出，保护系统安全的这些环节都有许多固有的缺陷和问题，但如果把这些话题当作已经讲完了，那么从安全的角度看，操作系统还有什么需要操心的呢？为什么这些还不是全部？

我们之所以需要更多东西，有好几个原因。特别重要的一点是：并非一切都在操作系统的控制之下。也许你会反驳：你告诉过我操作系统是无所不能的！其实不然。它只在一个有限的范围内拥有很大的控制权，这个范围就是它运行所在的硬件，以及它被授予控制权的那些接口。对其他机器上发生的事，它并没有真正的控制权；如果它的某个硬件部件被某种操作系统控制不了的机制访问了，它也管不了。

可是，如果系统本身并不控制对某个资源的访问，我们又怎能指望操作系统去保护它呢？答案是事先让资源做好应付麻烦的准备。实质上，我们假定数据一定会丢失，或者对手一定会试图非法篡改它，然后采取措施，确保这些行为不会给我们带来问题。关键的一点是：如果对手拿到的数据是一种他看不懂的形式，我们的秘密就是安全的。进一步说，如果攻击者看不懂这些数据，大概也就没法篡改它们，至少没法以可控的方式篡改。如果攻击者都不知道数据是什么意思，又怎么能把它改成自己想要的样子呢？

我们要用的核心技术是**密码学**（cryptography）。它是一组技术，能以可控的方式、按预期的结果，把数据从一种形式转换成另一种形式。我们会用密码学把数据从普通形式转换成另一种形式。如果做得对，对手通过研究受保护的形式，将无法判断原始数据是什么。当然，如果我们自己还想再用这些数据，就必须能逆转这种转换，把数据还原成普通形式。而这件事对手也必须很难做到。如果能做到这一步，我们还能为数据提供一些防篡改的保护，或者更准确地说，能阻止对手按自己的意愿篡改数据，甚至能知道对手何时动过我们的数据。这一切都要归功于密码学的妙处！

但要正确使用密码学并不容易，而且很多密码学用法的计算开销都很大。所以，在哪里、在什么时候使用密码学，我们要有所选择；怎样实现它、怎样把它集成到系统中，也要十分小心。选得好、做得对，安全性会大大提高；选得不好、实现得差，就毫无帮助，甚至可能有害。

关键问题：
如何保护操作系统控制范围之外的信息
即使其他人在操作系统的控制之外拿到了关键数据，我们怎样用密码学确保他们既无法使用、也无法篡改这些数据？有哪些密码学技术可以帮助解决这个问题？怎样正确地使用这些技术？用它们能做的事有哪些局限？

56.2 密码学

关于密码学的书已经有很多了，而我们只打算花一章的篇幅来讲它。好在密码学中有一些重要而复杂的问题我们大可不管，所以仍然能讲出一些有用的东西。这是因为我们自己并不打算成为密码学家，只是这项技术的使用者，依靠这个深奥领域里的专家给我们提供工具，我们不必完全理解这些工具的工作原理就能使用它们[1]。这听起来有点靠不住，可你其实早就在这么做了。真正懂得计算机硬件工作原理深层细节的人相对很少，但我们都能顺利地使用它，因为我们有好的接口，也知道是一群聪明人精心为我们打造了这些硬件。同样，密码学给我们提供了强大的接口、明确定义的行为，而且比一般情况更有把握：我们所用的工具背后凝聚了大量智慧。

话虽如此，密码学并不是魔杖，仅仅为了正确地使用它，你就有很多东西需要了解。这正是我们在这里要重点讲的内容，尤其是在操作系统的语境下。

密码学背后的基本思想是：拿一段数据，用一个算法（通常称为**密码算法**，cipher），通常再辅以第二段信息（称为**密钥**，key），把这段数据转换成另一种形式。新的形式应该和旧的形式毫无相似之处；但通常，我们希望能运行另一个算法，同样辅以第二段信息，把数据转换回原来的形式。

我们把这个过程稍微形式化一点。从数据 P（通常称为**明文**，plaintext）、密钥 K 和加密（encryption）算法 $E()$ 出发，我们最终得到 C，也就是 P 被改变后的形式，通常称为**密文**（ciphertext）：

$$C = E(P, K) \tag{56.1}$$

例如，我们可以把明文“Transfer $100 to my savings account”（转 100 美元到我的储蓄账户）转换成密文“Sqzmredq #099 sn lx rzuhmfr zbbntms”。这个例子其实用的是一种相当差劲的加密算法，叫作凯撒密码（Caesar cipher）。花一两分钟研究一下这段明文和密文，看看你能不能猜出这里用的是什么加密算法。

逆向转换以我们刚刚得到的 C、解密（decryption）算法 $D()$ 和密钥 K 为输入：

$$P = D(C, K) \tag{56.2}$$

于是，我们可以把“Sqzmredq #099 sn lx rzuhmfr zbbntms”解密回“Transfer $100 to my savings account”。如果你已经猜出了我们当初是怎样加密的，那么猜出怎样解密应该也很容易。

[1]如果你想多了解一些密码学引人入胜的历史，可以看看 Kahn 的书 [K96]。如果你想了解更多技术细节，Schneier 的书 [S96] 是个不错的起点。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

密码学的用途很多，但泛泛讨论时，人们通常讲的是发送和接收消息。在这样的讨论中，明文 P 是我们想发送的消息，密文 C 是这条消息受保护的版本，我们把它发送到冷酷无情的外部世界中去。

加密过程要有用，就必须是确定性的：第一种转换用某个特定的 K，总是把某个特定的 P 转换成某个特定的 C；第二种转换用这个特定的 K，总是把这个特定的 C 转换回原来的 P。很多情况下，$E()$ 和 $D()$ 其实是同一个算法，不过这并不是必须的。另外，在不知道 K 的情况下，应当很难从 C 推出 P。要是能做到完全不可能就最好了，不过我们通常满足于在计算上不可行。有了这个性质，我们就可以把 C 拿给世界上最充满敌意、最聪明的对手看，他们照样无法知道 P 是什么。当然，前提是……

就在这里，干净利落的理论论文开始和乱糟糟的现实发生碰撞。只有当对手并不同时知道 $D()$ 和我们的密钥 K 时，我们才能得到这种令人安心的保密性。如果这两样都被他们知道了，对手就会对 C 施用 $D()$ 和 K，提取出和我们一样的信息 P。

事实证明，我们通常没法对 $E()$ 和 $D()$ 保密。由于我们并不想当密码学家，这里就不深究原因了，不过设计出好的密码算法（cipher）是极其困难的。如果密码算法（cipher）有弱点，对手即使没有 K，也能提取出明文 P。所以我们需要一个非常好的密码算法（cipher），而这种算法很难得到。我们大多数人手边并没有一位世界级的密码学家来设计新算法，所以只能依赖为数不多的几种已知的强密码算法（cipher）。AES 是一种经过精心设计和深入研究的标准密码算法（cipher），是值得你考虑使用的一个好例子。

这听起来就像我们扔掉了一半的保护，因为现在密码学的好处完全取决于密钥的保密。正是如此。这一点太重要了，你必须牢牢记住，所以我们加粗再说一遍：**密码学的好处完全取决于密钥的保密**。把这句话再读几十遍大概也没有坏处，因为到处都是没把这一教训放在心上、因而不安全的系统。

好消息是，如果你用的是强密码算法（cipher），并且小心地为密钥保密，你的密码学就是牢靠的，别的什么都不用担心。坏消息是，在密码学的实际应用中，要在现实的系统里为密钥保密并不容易。这一点我们后面还会多讲。

此刻，先为我们取得的保护高兴一下吧，而且还有更值得欢喜的：正确地使用密码学，带给我们的不只是保密性！想想我们所做的这些转换有什么性质。如果对手拿到了我们加密后的数据，他看不懂。可要是对手能篡改它呢？他篡改的是加密后的形式，也就是对 C 做了些改动，把它变成了比如说 C'。我们试图解密 C' 时会发生什么？它不会解密成 P，而会解密成别的东西，比如说 P'。对于你应该使用的那类好的密码算法（cipher）来说，除非知道 K，否则很难确定一段密文 C' 会解密成什么。这意味着，很难预测需要哪段密文才能解密出某段特定的明文；而这又意味着，攻击者完全不知道被他改过的密文 C' 会解密成什么。

在它可能解密成的所有位模式中，结合我们预期看到的内容（ASCII 文本、格式正确的 PDF 文件，或者别的什么）来看，P' 很可能是一堆垃圾。如果我们足够小心，就能发现 P' 并不是我们最初的数据，从而知道对手篡改过我们加密的数据。如果想更有把握，可以对明文执行一次散列函数（hash function），把散列值（hash）附在消息或加密文件里一起发送。如果解出来的明文算不出同样的散列值，就有充分的迹象表明出了问题。

所以，我们也可以用密码学来帮助保护数据的完整性（integrity）。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：自己开发密码算法——别这么做

别这么做。

我们很想就此打住，因为遵守这条建议真的很重要。但你也许不信，所以我们再多解释几句。世界上最好的密码学家设计出的密码算法（cipher）也常常有缺陷。你是世界上最好的密码学家之一吗？如果不是，而顶尖专家都常常造不出强密码算法（cipher），你凭什么认为自己能做得更好，甚至做得一样好？

我们知道你接下来会说什么："可我写的密码算法（cipher）强得连我自己都破解不了。"嗯，几乎任何人只要用心，都能造出一个自己破解不了的密码算法（cipher）。可还记得我们说过的那些世界级的密码学家吗？他们是怎样成为世界级的？靠仔细研究密码学的基础，靠破解别人的密码算法（cipher）。他们非常擅长这件事；如果值得费这个劲，他们就会破解你的算法。如果你只是到处吹嘘自己的密码算法（cipher）有多妙，他们也许懒得理你（因为这种话他们听得太多了）；可一旦你真把它用在什么重要的事情上，很不幸，你就会引起他们的注意。接着，你的秘密就会被揭穿；再接着，你就会因为自己设计密码算法（cipher）、而不用 AES 这样的标准算法而显得很傻，何况用标准算法本来就更省事。

所以，别这么做。

等等，还有呢！如果有人交给你一段数据，它是用只有你和你的好友 Remzi 知道的密钥 K 加密的，会怎样？你知道这段数据不是你造的，所以如果用密钥 K 能正确地解密它，你就知道它一定是 Remzi 造的。毕竟，他是唯一另一个知道密钥 K 的人，所以只有他能完成这次加密。瞧，我们就这样用密码学实现了身份认证！遗憾的是，密码学不会帮你打扫房间、替你写作业，也不能在几秒钟内切出几千根薯条，但无论如何，它都是一个非常好的工具。

我们刚才描述的这种密码学通常称为**对称密码学**（symmetric cryptography），因为加密和解密用的是同一个密钥。很长一段时间里，所有人都认为这是唯一可能的密码学形式。结果证明，所有人都错了。

56.3 公钥密码学

讨论用密码学做身份认证时，你可能已经注意到了一个小问题：为了验证一段加密信息的真实性（authenticity），你需要知道加密它时所用的密钥。如果我们只关心用密码学做身份认证，这就很不方便。这意味着，我们必须把为此所用的密钥告诉每一个可能需要认证（authentication）我们的人。假如我们是微软，想向每个买了我们软件的用户证明自己的身份，该怎么办？我们不能只用一个密钥来做这件事，因为那样就得把这个密钥发给数以亿计的用户；而他们一旦有了这个密钥，就可以用它加密信息，冒充微软。或者，微软也可以为这数以亿计的用户每人生成一个不同的密钥，但那样就得秘密地把各不相同的密钥分别交给数以亿计的用户，更不用说还要管理所有这些密钥。真是糟糕。

幸运的是，我们的好朋友，那些密码学奇才，想出了一个解决办法。如果密码学用两个不同的密钥，一个用来加密，另一个用来解密，会怎样？我们的加密操作变成

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

$$C = E(P, K_{encrypt}) \tag{56.3}$$

解密操作变成

$$P = D(C, K_{decrypt}) \tag{56.4}$$

微软的日子一下子好过多了。他们可以把解密密钥 $K_{decrypt}$ 告诉所有人，而对加密密钥 $K_{encrypt}$ 保密。现在，他们可以用自己的秘密密钥加密数据，以此证明数据的真实性，而他们数以亿计的用户可以用微软公开的密钥检验真实性。例如，微软可以用 $K_{encrypt}$ 加密一个操作系统更新，然后把它发给所有用户。每个用户都可以用 $K_{decrypt}$ 解密它。如果解密出来的是一个格式正确的软件更新，用户就可以确定它是微软制作的。由于没有其他人知道那个私钥（private key），别人不可能造出这个更新。

这听起来像魔法，但并不是。实际上，是数学来解救了我们，它常常这样。我们不在这里深入细节，但你得承认这相当巧妙。这种密码学称为**公钥密码学**（public key cryptography），因为两个密钥中有一个可以广为人知，向全体公众公开，同时仍能达到我们想要的效果。人人都知道的那个密钥称为**公钥**（public key），只有所有者知道的那个密钥称为**私钥**（private key）。公钥密码学（通常缩写为 **PK**）的发明史很复杂，虽然有趣，但和我们的讨论关系不大。想了解细节，可以看看这个领域的先驱 Whitfield Diffie 写的一篇论文 [D88]。

公钥密码学避开了早期各种密码学面临的一个难题：安全地分发秘密密钥。在这里，私钥由一方创建并由他保密，从不分发给任何其他人。公钥必须分发出去，但一般来说，我们并不在乎有第三方知道这个密钥，因为他们无法用它给消息签名（signature）。分发公钥比分发秘密密钥要容易，不过，唉，它比听起来要难。这一点我们后面会讲到。

公钥密码学其实还更巧妙，因为它反过来也行。你可以用解密密钥 $K_{decrypt}$ 加密，这时就需要用加密密钥 $K_{encrypt}$ 来解密。我们仍然希望加密密钥保密、解密密钥公开，所以按这个顺序做，就无法再实现身份认证了。毕竟，任何人都可以用 $K_{decrypt}$ 加密。但只有密钥的所有者才能用 $K_{encrypt}$ 解密这样的消息。所以，只要知道对方的公钥，任何人都可以给持有私钥的人发送加密消息。因此，用私钥加密，PK 可以实现身份认证；用公钥加密，PK 可以实现保密通信。

如果你两者都想要呢？你完全可能想要。这时就需要两对不同的密钥。假设 Alice 想用 PK 和她的朋友 Bob 秘密通信，同时还想确保 Bob 能认证她的消息。再假设 Alice 和 Bob 各有一对自己的 PK 密钥，每人都知道自己的私钥和对方的公钥。如果 Alice 用自己的私钥加密消息，她就认证了这条消息，因为 Bob 可以用她的公钥解密，并且知道只有 Alice 才能造出这条消息。可是每个人都知道 Alice 的公钥，所以这样做无法保密。不过，如果 Alice 把这条经过认证的消息再加密一次，这一次用 Bob 的公钥，她就同时实现了保密。只有 Bob 知道对应的私钥，所以只有 Bob 能读这条消息。当然，Bob 需要解密两次，先用他的私钥，再用 Alice 的公钥。

这听起来开销很大。实际上比你想的还糟，因为原来公钥密码学有一个缺点：它的计算开销比依赖单一共享密钥的传统密码学大得多。公钥密码学执行起来可能比标准的对称密码学慢几百倍。因此，我们实在负担不起什么都用公钥密码学。我们需要挑选使用它的地方，用它来做它擅长的事。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

还有一个重要的问题。我们相当轻率地说 Alice 知道 Bob 的公钥、Bob 知道 Alice 的公钥。这种美好的状态是怎么实现的？起初，只有 Alice 知道她的公钥，也只有 Bob 知道他的公钥。如果想享受公钥密码学的魔力，我们就得做点什么，把这些信息传给世界上的其他人。而且我们最好小心行事，因为 Bob 会认定，用他认为属于 Alice 的公钥加密的消息，确实是 Alice 造的。如果某个邪恶的天才（不妨叫她 Eve）设法让 Bob 相信 Eve 的公钥其实属于 Alice，会怎样？如果发生这种事，Bob 就会把 Eve 造的消息误认为来自 Alice，我们认证消息的整个目标就落空了。我们最好确保 Eve 无法在“哪个公钥属于 Alice”这件事上骗过 Bob。

这就引出了一条漫长而幽暗的道路，通向密钥分发（key distribution）基础设施这个神秘的领域。你最好别自己去走这条路，因为即使是准备最充分、敢于冒险上路的先驱，也常常落得个悲惨的下场。实践中我们怎样分发公钥，会在分布式系统（distributed system）安全一章中讨论。眼下，只要记住：公钥密码学美妙的魔力，建立在密钥分发（key distribution）这个脏兮兮、靠不住的基础之上。

关于 PK 密码学还有一件事：**密码学的好处完全取决于密钥的保密**。（我们打赌你以前听过这句话。）在这里指的是私钥。对公钥密码学的整体好处来说，私钥的保密和对称密码学中单一共享密钥的保密同样重要。绝不要泄露私钥。绝不要共享私钥。使用和存放私钥时要格外小心。如果私钥丢了，你用它做过的一切都会面临风险，谁拿到它，谁就能冒充你、读你的秘密消息。那可就太糟了，不是吗？

56.4 密码学散列

我们前面讨论过，可以用密码学保护数据的完整性，因为加密数据被改动后就无法正确解密。我们可以先对数据做散列（hash），只加密散列值，而不是加密整份数据，以此降低这种完整性检查的开销。然而，如果真想万无一失，我们就不能随便用一个散列函数，因为散列函数的本性决定了它们会有**散列冲突**（hash collision），也就是两个不同的位模式散列出同样的结果。如果攻击者能把我们打算发送的位模式换成另一个散列结果相同的位模式，我们就失去了完整性。

所以，为了格外小心，我们可以用**密码学散列**（cryptographic hash）来确保完整性。密码学散列（cryptographic hash）是一类特殊的散列函数，具有几个重要的性质：

- 要找到产生同样散列值的两个输入，在计算上不可行。
- 对输入的任何改动，都会使得到的散列值发生不可预测的变化。
- 仅凭散列值推断输入的任何性质，在计算上不可行。

基于这些性质，如果我们只关心数据完整性（data integrity）而不关心保密性，就可以对一段数据求密码学散列（cryptographic hash），只加密这个散列值，然后把加密后的散列值和未加密的数据一起发给对方。如果对手在传输途中动了数据，我们解密散列值、再对数据重做一遍散列操作时，就会发现两者不匹配，从而察觉到篡改[2]。

[2]为什么要加密密码学散列（cryptographic hash）值呢？因为任何人，包括我们的对手，都可以对任何东西运行密码学散列（cryptographic hash）算法，包括被改动过的消息版本。如果我们不加密散列值，攻击者就会修改消息，算出一个新的散列值，用这两样替换原来的消息和散列值，再把结果发出去。但如果我们发送的散列值是加密的，攻击者就无法知道改动后的散列值加密后应该是什么样。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

把它稍微形式化一点：要执行密码学散列（cryptographic hash），我们取一段明文 P 和一个散列算法 $H()$。注意，这里不一定涉及任何密钥。过程是这样的：

$$S = H(P) \tag{56.5}$$

由于密码学散列（cryptographic hash）是一般散列的一个子类，我们通常期望 S 比 P 短，也许短很多。这意味着会有冲突，也就是两段不同的明文 P 和 P' 都散列成 S 的情况。然而，上面列出的密码学散列（cryptographic hash）的性质会让对手很难利用冲突。即使你同时知道 S 和 P，也应当很难找到另一段散列成 S 的明文 P'[3]。对于改动后的明文 P'，要算出它的 S' 并不难，因为直接对 P' 运行密码学散列（cryptographic hash）算法就行了。但即使对 P 只做极小的改动，比如只差一位的 P'，算出的散列值 S' 也应当和 S 有完全不可预测的差别。

密码学散列（cryptographic hash）除了确保加密数据的完整性，还有别的用途。例如，正如身份认证一章所讨论的，存储加盐（salting）散列的密码时，首选的就是这类散列。它们可以用来判断存储的文件是否被改动过，Tripwire 这类著名的安全软件提供的就是这项功能。它们还可以用来迫使进程（process）在提交请求之前完成一定量的工作，这种方法称为“工作量证明”（proof of work）。提交者必须提交一个请求，用某个指定的密码学散列（cryptographic hash）算出的值要等于某个特定的值；由于这类散列的性质，提交者必须尝试大量的请求格式，才能找到一个散列成所需值的格式。每次散列操作都要花些时间，所以提交一个合格的请求需要一定量可预测的工作。散列的这种用法以各种形式出现在好几类应用中，包括防垃圾邮件和区块链。

和其他密码学算法一样，密码学散列（cryptographic hash）最好也使用标准算法。例如，SHA-3 算法通常被认为是一个好的选择。不过，密码学散列（cryptographic hash）算法过时的事屡见不鲜，所以如果你在设计一个用到这类算法的系统，明智的做法是先查一查目前推荐使用的是哪些算法。

56.5 破解密码学

你很可能听说过有人破解密码学。这是影视作品里很流行的题材。你该有多担心呢？

嗯，如果你没听我们前面的劝告，自己造了一个密码算法（cipher），那你就该非常担心。担心到应该马上停止阅读，把你自己的密码算法（cipher）从系统里拆掉，换上一个知名的、受人认可的标准算法。去吧，你回来时我们还在这儿。

如果你用的是那些标准算法之一呢？那你多半没事。如果你用的是现代标准算法，除了少数无关紧要的例外，目前还没有已知的方法能在不拿到密钥的情况下读取用这些算法加密的数据。这并不是说你的系统就安全了，而是说大概没人会通过破解密码学算法来攻进你的系统。

那他们会怎么做呢？多半是利用你系统中与密码学毫不相干的软件缺陷；不过，也有一定可能是通过拿到你的密钥，或者利用你在密码学管理上的其他缺陷来破解它。怎么做到呢？在创建和使用密钥的方式上存在软件缺陷，这是一个常见问题。在分布式环境中，

[3]每隔一段时间，就会有某个知名的密码学散列函数（cryptographic hash function）被“攻破”，意思是有人想出了办法，能造出一个 P'，用这个函数算出和 P 相同的散列值。2017 年，一个名为 SHA-1 的散列函数就遭遇了这种事，这个函数因此变得不安全，不能再用于完整性保护 [G17]。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

共享密钥所用方法中的缺陷也是一个可被利用的常见弱点。Peter Gutmann 对密码学管理不当经常引发的各类问题做过一个很好的调查 [G02]。例子包括：在许多人共用的软件里分发秘密密钥；错误地在网络上传输密钥的明文版本；从一个大大缩小了的候选集合里选择密钥，而不是从理论上可能的更大集合里选。最近，Heartbleed 攻击展示了一种方法：从远程计算机的内存中获取 OpenSSL 会话所用的密钥，从而让攻击者解密整个会话；而密码算法（cipher）本身、它的实现以及它的密钥选择过程，都没有任何缺陷。攻击者借助这个漏洞，能读取所有使用 HTTPS（HTTP 的密码学保护版本）的网站中四分之一到一半网站的流量 [D+14]。

攻击者对付密码学的一种办法是猜密钥。这样做其实根本没有破解密码学。密码学算法的设计目的，是防止不知道密钥的人获取秘密。如果你知道密钥，解密本来就不应该难。

所以，攻击者可以简单地把每个可能的密钥都猜一遍、试一遍。这称为暴力破解（brute force attack），这也是你应该使用长密钥的原因。例如，AES 密钥至少有 128 位。假设你的 AES 密钥是随机生成的，攻击者平均要猜 2^{127} 次才能猜中。猜这么多次要花很多时间。当然，如果某个软件缺陷导致你的系统只从 32 个可能的 AES 密钥中选一个，而不是从 2^{128} 个中选一个，暴力破解可能就变得易如反掌了。密钥的选择对密码学来说是件大事。

例如，最初的 802.11 无线网络标准根本没有对在空中传输的数据做任何密码学保护。

为它增加这种保护的第一次尝试叫作 WEP（Wired Equivalent Protocol，有线等效协议，这个名字相当乐观）。WEP 受制于必须塞进现有标准，但它生成和分发对称密钥的方法有严重缺陷。攻击者只要在一个 802.11 网络上监听无线流量，最快一分钟内就能确定正在使用的密钥。有一些随处可得的工具，任何人都能用它们做到这一点[4]。

提示：选择密钥

密钥保密的一个重要方面，是从一开始就选一个好的密钥。对公钥密码学来说，你需要运行一个算法，从为数不多的几种可能的密钥对中选出你要用的那一对。但对对称密码学来说，你可以从所有可能的密钥中随意挑一个。该怎么挑呢？

随机挑。如果你用任何确定性的方法选密钥，对手要找出你的密钥这个问题，就变成了要搞清楚你的方法这个问题。更糟的是，由于你多半会在一段时间里生成很多密钥，他一旦知道了你的方法，就能拿到所有密钥。而如果你用随机的方式生成密钥，对手搞清楚其中一个，也帮不了他推出你的其他任何密钥。密码系统的这个非常理想的性质称为**完美前向保密**（perfect forward secrecy）。

遗憾的是，真正的随机性（randomness）很难得到。就操作系统而言，最好的来源是考察那些被认为本质上随机的硬件过程，比如硬件完成操作所需时间的低位，再把结果转换成随机数。这称为**收集熵**（gathering entropy）。在 Linux 中，系统会自动替你做这件事，你可以读 /dev/random 来使用收集到的熵。Windows 也有类似的收集熵的功能。用它们来生成你的密钥。它们并不完美，但对许多用途来说已经足够好了。

[4]WEP 后来被 WPA 取代了。不幸的是，WPA 也有自己的弱点，于是又被 WPA2 取代。不幸的是，WPA2 也有自己的弱点，所以从 2018 年起，它正在被 WPA3 取代。为无线网络提供密码学保护的这段悲惨经历，应当给那些想低估把这件事做对有多难的人一个教训。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

再举一个例子：Netscape 浏览器的早期实现，用一些很容易猜到的值（比如当天的时间和请求密钥的进程的 ID）作为随机数生成器的种子来生成密码学密钥。研究人员发现，他们大约 30 秒就能猜出生成的密钥 [GW96]。

你可能听说过 PK 系统用的密钥要长得多，2K 或 4K 位。听起来安全多了，不是吗？这至少能让它们更能抵御暴力破解吧？然而，这类密码系统的密钥不能随机选择。可能的公钥和私钥对只有相对很少的几种。这是因为公钥和私钥必须相互关联，系统才能工作。这种关联通常是数学上的，而且通常故意设计成在数学上很难推导，所以知道公钥并不会让人很容易得知私钥。但是，有了公钥，人们最终可以利用这个系统的数学性质推出私钥。这就是 PK 系统要用这么大的密钥的原因：确保这个“最终”是很长很长的时间。

但只有你对私钥保密，这一点才有意义。到现在，我们希望这听起来已经是显而易见的了；可许多嵌入式设备的制造商用 PK 为这些设备提供加密，却把私钥放进了设备的软件里。而且很多时候，同一型号的所有设备都用同一个私钥。这样共用的私钥，无一例外都会变得，嗯，公开。2016 年 9 月，一项研究发现有 450 万台嵌入式设备依赖的私钥已经不再私密了 [V16]。任何人都可以出于任何目的冒充这些设备中的任何一台，也可以读取任何用 PK 发给它们的信息。实质上，这些设备所做的密码学不过是装点门面，并没有让设备的安全性有任何明显的提升。

总之，破解密码学通常就是想办法得知密钥。或者，你可能已经猜到了：**密码学的好处完全取决于密钥的保密**。

56.6 密码学与操作系统

密码学引人入胜，但引人入胜的东西多了[5]，很多和操作系统毫无关系。我们为什么要费心花半章的篇幅讲密码学？因为我们可以用它来保护操作系统。

但并不是在任何地方、为任何目的都能用。我们已经反复向你灌输：要有效地使用密码学，密钥保密至关重要。这应该让你明白，只要密钥无法保密，就无法有效地使用密码学。回想一下关于安全的第一章：操作系统控制着计算机上的所有资源，也能访问它们。这意味着，如果你在计算机上有加密的信息，而解密它所需的密钥也在同一台计算机上，那么这台机器上的操作系统就能解密这些数据，不管这是不是你想要的结果[6]。

你要么信任你的操作系统，要么不信任。如果不信任，日子反正不会好过；其中一个后果是，这个不可信的操作系统一旦接触过你的秘密密钥，就可以复制它，随时重复使用。反过来，如果你信任你的操作系统，就不需要对它隐藏你的数据，这时也就用不着密码学。这一点适用于任何你把数据交给不信任的东西的场合。例如，如果你在数据方面不信任你的云计算服务商，那么把明文数据交给他们、再请他们加密，并不会让情况有任何改善。他们已经看过明文了，还可以留一份密钥的副本。

如果你确信你的操作系统现在可信，但担心它将来可能不可信，你可以现在就加密某些东西，并确保密钥不存放在这台机器上。当然，如果你对操作系统当前的安全性判断错了，

[5]比如贝多芬晚期的钢琴奏鸣曲。他最后一首奏鸣曲（作品 111 号）中的一个乐章，听起来甚至像爵士乐，而它写于 19 世纪 20 年代！

[6]不过，还记得前面某一章中讨论过的安全飞地（security enclave）吗？那种硬件不允许操作系统完全访问飞地所保护的信息。想一想，对于使用这种飞地的计算机，这对密码学意味着什么，又带来了哪些新的可能。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

或者在操作系统变坏之后，你又在这台机器上解密过这些数据，那么你的密码学就保护不了你，因为至关重要的密钥保密已经被破坏了。

有人会说，操作系统被攻陷并不总是永久性的。很多是，但有些只让攻击者暂时访问系统资源，或者只能访问少数几样特定资源。在这些情况下，如果加密数据没有以明文存储，而且在攻击者能访问的时间或地点拿不到解密密钥，那么加密这些数据可能仍然有好处。这里棘手的问题在于，你无法事先知道对你系统的成功攻击是不是只会发生在特定的时间、持续特定的时长，或者只针对系统中的特定部分。所以，如果你采用这种方法，就要尽量减少一切暴露：少解密，迅速而谨慎地处理掉明文数据，除了执行密码学操作的时候，不在系统中保留密钥的明文版本。这样的最小化可能很难做到。

如果密码学无法完全保护我们免受不诚实的操作系统的侵害，那么操作系统还能拿密码学做什么用呢？我们在身份认证一章中看到过一个特殊的例子。有些密码学操作是单向的：能加密，但永远无法解密。即使操作系统被攻陷，我们也可以用这类操作以加密形式安全地存储密码，因为加密后的密码无法被解密[7]。

还有什么用途？在分布式环境中，如果我们在一台机器上加密数据，然后通过网络发出去，那么所有中间组件都不属于我们的机器，因而拿不到密钥。数据在传输途中就受到了保护。当然，最终目的地机器上的那位同伴，要使用这些数据也需要密钥。我们之前承诺过，这个问题会在另一章中讨论。

还有吗？嗯，如果有人能不经过我们的操作系统就访问到我们的某些硬件，会怎样？如果存放在这些硬件上的数据是加密的，而密钥又不在这个硬件上，密码学就能保护这些数据。这种形式的加密有时称为**静态数据加密**（at-rest data encryption），以区别于对在机器之间传输的数据进行加密。它很有用，也很重要，所以我们来详细看看。

56.7 静态数据加密

我们在持久性（persistence）部分的各章中看到，数据可以存放在磁盘驱动器（disk drive）、闪存盘（flash drive）或其他介质上。如果是敏感数据，我们可能希望给它加上一些想要的安全属性，比如保密性或完整性。为这些数据实现这些目标的一种技术，是以加密形式而不是明文形式存储它们。当然，加密数据无法用于大多数计算，所以如果存放数据的机器需要对数据做一般的计算，就必须先解密[8]。如果目的只是保存一份安全的数据副本，而不是使用它，那也许不需要解密，但这并不是常见的情况。

数据可以用不同的方式加密：使用不同的密码算法（DES、AES、Blowfish），采用不同的粒度，比如记录、数据块、单个文件或整个文件系统（file system）；由不同的系统组件完成，比如应用程序、库、文件系统或设备驱动程序（device driver）。静态数据加密（at-rest data encryption）的一种常见的通用用法叫作**全盘加密**（full disk encryption）。这通常意味着存储设备（storage device）的全部内容（或几乎全部内容）都被加密。虽然名字叫

[7]但是，唉，只要合法用户曾经向被攻陷的操作系统提供过正确的密码，一切就都完了。被攻陷的操作系统会在用单向密码学散列（cryptographic hash）算法处理密码之前，复制下用户提供的密码，交给幕后的某个坏蛋。

[8]有一个可能的例外值得一提。那些密码学奇才发明了一种叫作**同态加密**（homomorphic cryptography）的密码学，它允许你在不解密的情况下，对数据的加密形式进行运算。例如，你可以在不先解密的情况下，给一个加密的整数加一。等你解密结果时，果然，原来的数字就加了一。同态密码算法（cipher）已经开发出来了，但截至本章写作时，高昂的计算和存储开销让它们在大多数场合都不实用。也许随着时间推移，这会有所改变。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

全盘加密，但它实际上可以用在许多种持久存储介质上，而不只是硬盘驱动器（hard disk drive）。全盘加密通常要么由硬件提供（内置在存储设备中），要么由系统软件提供（设备驱动程序或文件系统的某个部分）。无论哪种情况，操作系统都在所提供的保护中扮演着一定的角色。Windows 的 BitLocker 和苹果的 FileVault 就是基于软件的全盘加密的例子。

一般来说，在启动时，系统会向用户索要解密密钥，或者索要可以用来获取密钥的信息（比如密码短语，passphrase，它类似于密码，但可能由多个单词组成）。如果提供的信息正确，硬件或操作系统就能拿到执行解密所需的一个或多个密钥。数据放到设备上时，会被加密；数据从设备上取出时，会被解密。只要数据还存放在机器内存（machine memory）中的任何地方，包括共享缓冲区（buffer）或用户地址空间（address space）中，它就一直保持解密状态。新数据要发送到设备时，会先被加密。数据绝不会以解密的形式放到存储设备上。在最初那次获取解密密钥的请求完成之后，加密和解密对用户和应用程序完全透明。他们永远看不到加密形式的数据，在机器重启之前，也不会再被要求提供密钥。

密码学是一种计算开销很大的操作，用软件来做尤其如此。执行基于软件的全盘加密会带来一定的开销。各方报告的开销大小不一，但对磁盘密集型操作来说，多出百分之几的延迟是常见的。对于不怎么用磁盘的操作，开销可能察觉不到。对于基于硬件的全盘加密，磁盘驱动器能达到它的额定速度，这个速度可能比不使用全盘加密的同类型号慢，也可能不慢。

这种形式的加密能防范什么呢？

- 对于试图访问自己不该看到的数据的用户，它没有提供额外的保护。操作系统提供的标准访问控制机制要么有效（这样的用户因为没有访问权限而拿不到数据），要么无效（这时这样的用户和其他任何人一样，可以使用解密密钥）。
- 它不能防范泄露数据的应用程序缺陷。这类缺陷会让攻击者冒充用户，所以如果用户能访问未加密的数据，攻击者也能。例如，它对缓冲区溢出（buffer overflow）或 SQL 注入几乎没有什么保护作用。
- 它不能防范系统上不诚实的特权用户，比如系统管理员。管理员的特权可能让他冒充拥有这些数据的用户，或者安装能访问用户数据的系统组件；所以，管理员可以按需访问数据的解密副本。
- 它不能防范操作系统本身的安全缺陷。一旦提供了密钥，操作系统就能使用它（直接在内存中使用，或者间接地请硬件使用它），不管这个操作系统是可信而安全的，还是已被攻陷而不安全的。

那么，这种形式的加密有什么好处呢？考虑这样一种情况。如果一个存储数据的硬件设备被物理地从一台机器移到另一台机器上，另一台机器上的操作系统没有义务遵守设备上存储的访问控制（access control）信息。实际上，它甚至不必用同一个文件系统来访问这个设备。例如，它可以把这个设备仅仅当作一个原始数据块的来源，而不是一个组织好的文件系统。所以，新的操作系统可能会无视与设备上的文件相关联的任何访问控制信息。

然而，如果设备上的数据用全盘加密加过密，新机器通常就拿不到加密密钥。它可以访问原始数据块，但这些块是加密的，没有密钥就无法解密。例如，如果相关硬件被偷走并接到另一台机器上，这个好处就派上用场了。对于经常丢失或被盗的移动设备来说，这种情况是非常现实的可能。磁盘驱动器有时会被转卖，而再次购买的人在上面发现过属于原主人的数据（包括相当敏感的数据）。在这些重要的情况下，全盘加密确实能带来实实在在的好处。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

对于其他形式的静态数据加密（at-rest data encryption），系统仍然必须解决这些问题：加密多少数据，怎样获取密钥，以及何时加密和解密数据。这些问题的处理方式不同，得到的保护类型也不同。一般来说，这些情况要求有某个软件确保数据的未加密形式不再存放在任何地方，包括缓存（cache），并确保那些可能试图非法访问数据的人拿不到密码学密钥。这种保护有价值的场合相对较少，但也有几个常见的例子：

- 归档那些可能需要复制、必须保存、但不必使用的数据。这时，数据可以在创建时就加密，也许永远不解密，或者只在数据所有者控制下的特殊情况下才解密。如果数据最初加密时机器没有被攻陷，而且密钥不永久存放在系统中，加密数据就相当安全。不过要注意，如果密钥丢了，你就再也无法解密这些归档数据了。
- 在云计算设施中存储敏感数据，这是上一个例子的变体。如果你不完全信任云计算服务商（或者不确定这家服务商有多谨慎；记住，信任另一个计算组件时，你信任的不仅是它的诚实，还有它的谨慎和正确性），那么在把数据发到云设施之前先加密，是明智的做法。许多云备份产品都包含这项功能。这时，密码学操作和密钥的使用，都发生在数据移到不可信的系统之前，或者从那个系统取回之后。
- 由应用程序执行的用户级加密。例如，用户可能选择加密一封电子邮件，存储下来的任何版本都是加密形式。这时，密码学操作由应用程序执行，用户要做点什么，把密码学密钥提供给应用程序。理想情况下，加密完成后，应用程序会确保数据的未加密形式和加密所用的密钥都不再轻易可得。不过要记住，只要密钥还存在，操作系统就能在你的应用程序不知情的情况下访问它。

加密选定的静态数据，有一个重要的特殊情况，就是我们在身份认证一章中讨论过的**密码库**（password vault，也称 **key ring**）。典型的用户要和许多远程网站打交道，这些网站都要求他们提供密码，还记得吗，这是基于“你知道什么”的身份认证。每个网站用不同的密码，安全性最好，但这会给人类用户带来负担，因为人一般很难记住许多密码。一个解决办法是把所有不同的密码都加密存放在机器上，按使用它们的网站编索引。需要某个密码时，就把它解密，提供给需要它的网站。

对于密码库（password vault）和所有这类特殊情况，每当需要加密或解密数据时，系统都必须有办法拿到所需的密钥。如果攻击者能拿到密钥，密码学就毫无用处了，所以安全地存放密钥变得至关重要。一般来说，如果密钥以未加密的形式存放在这台计算机上的任何地方，加密数据就面临风险，所以设计良好的加密系统往往不这么做。例如，对于密码库（password vault），用来解密密码的密钥并不存放在机器的稳定存储（stable storage）中。需要时，系统会向用户索要这个密钥，或者索要用来推导出密钥的密码短语。然后用这个密钥解密所需的密码。要做到最安全，就应该在完成这次解密后立即销毁密钥——还记得最小特权（least privilege）原则吗？但这样做意味着每次需要密码时，用户都得重新输入密钥——还记得可接受性（acceptability）原则吗？在大多数情况下，人们在易用性和安全性之间取了个折中：用户第一次输入密钥后，系统在相当长的一段时间里记住它，但只把它保存在 RAM 中。当用户注销、系统关机，或者负责密码库（password vault）的应用程序（比如 Web 浏览器）退出时，密钥就被“忘掉”了。这种方法让人联想到单点登录（single sign-on）系统：用户第一次访问系统时要输入密码，此后直到注销都不需要再次认证。它的缺点也和那些系统一样，比如无人看管的终端可能被未经授权（authorization）的人用来冒用

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

别人的访问权限。两者都有一个巨大的优点：它们不会把用户烦得干脆弃之不用，转投毫无安全性可言的系统。

56.8 密码学能力

还记得访问控制一章里讲过，能力有一个问题：我们不能把它们留在用户手里，否则用户就可以伪造能力，给自己授予任何想要的访问权。密码学可以用来创建不可伪造的能力。一个可信实体可以用密码学创建一个足够长、加密足够安全的数据结构（data structure），表明持有者获准访问某个特定资源。然后可以把这个数据结构交给用户，用户把它出示给相应资源的所有者，以获得访问权。实际控制资源的系统必须能在批准访问之前检查这个数据结构是否有效，但不需要维护访问控制列表（access control list）。

这样的密码学能力可以用对称密码学创建，也可以用公钥密码学创建。用对称密码学时，能力的创建者和检查它的系统需要共享同一个密钥。当这两个实体是同一个系统时，这种做法最可行；否则就需要在需要使用密钥的机器之间传递密钥，而根据使用场景，可能还要高速、大规模地传递。你可能会好奇：单独一台机器为什么要费事去创建一个密码学能力来允许访问，而不是简单地记住用户已经通过了访问检查？其实有几个可能的原因。例如，如果控制资源的机器要和数量巨大的用户打交道，跟踪每个用户的访问状态会代价高昂而又复杂，在系统需要操心故障和延迟的分布式环境中尤其如此。或者，如果系统希望访问权可以转移，比如主体可能从一台机器换到另一台机器，那么让能力随主体一起移动，在任何地点都能使用，会更可行。如果创建和检查能力的所有机器本来就是可信的，密钥分发（key distribution）也不成问题，那么对称密码学能力同样说得通。

如果用公钥密码学创建能力，创建者和资源控制者就不必在同一个地方，信任关系也不必那么强。能力的创建者需要其中一个密钥，通常是秘密密钥，资源的控制者需要另一个。如果能力的内容本身不是秘密，就可以使用真正的公钥，不必在意谁知道它。如果需要保密（或者至少某种程度的隐蔽），那么本来可以作为公钥的那个密钥，可以只分发给需要检查能力的有限几个实体[9]。资源管理者可以创建一组凭证（表明哪个主体获准以哪些方式、在多长时间内使用哪些资源），然后用私钥加密它们。其他任何人都可以用管理者的公钥解密这些凭证（credential）来验证它们。只要只有资源管理者知道私钥，就没有人能伪造能力。

如上所述，这样的密码学能力可以容纳相当多的信息，包括过期时间、获得这个能力的一方的身份，以及其他许多信息。由于强密码学能确保所有这些信息的完整性，能力是可以信赖的。这一特性让能力的创建者至少能在一定程度上防止能力被任意复制和共享。例如，用在网络环境中的密码学能力可以与某个特定的 IP 地址绑定，只有当携带它的消息来自那个地址时，才被认为有效。

可以使用许多不同的加密方案。重要的一点是，加密后的能力必须足够长，使得通过暴力枚举或随机猜测找到一个有效能力在计算上不可行（例如，无效位模式的数量是有效位模式数量的 10^{15} 倍）。

关于密码学能力，我们会在分布式系统安全一章中再多讲一些。

[9] 不过要记住，如果你把一个密钥嵌在一个广泛分发的软件里，那这个密钥肯定会变得人尽皆知。所以，即使你认为对应的那个密钥是秘密的、而不是公开的，也不宜过分依赖这种想法。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

56.9 小结

即使数据已经不在系统的保管之下，密码学也能为它提供某些形式的保护。这些保护形式包括保密性、完整性和身份认证。密码学通过一种称为密码算法（cipher）的算法，把数据原来的位模式转换成另一种位模式，从而实现这种保护。大多数情况下，这种转换可以逆转，得到原来的位模式。对称密码算法（cipher）使用一个秘密密钥，由所有有权访问这些数据的各方共享。非对称密码算法（cipher）用一个密钥加密数据，用另一个密钥解密数据，其中一个密钥保密，另一个通常公开。而密码学散列（cryptographic hash）不允许逆转，也不需要使用密钥。

强密码算法（cipher）使得在拿不到所需密钥的情况下获取原来的位模式在计算上不可行。对于对称和非对称密码算法（cipher），这意味着只有持有正确密钥的人才能获得密码算法带来的好处。由于密码学散列（cryptographic hash）没有密钥，这意味着任何人都不应该能从散列值得到原来的位模式。

对操作系统来说，密码学能派上用场的明显场合，是数据要发送到另一台机器上，或者存放数据的硬件可能在没有操作系统介入的情况下被访问。对于后一种情况，数据可以在设备上加密（用硬件或软件），在交给操作系统时解密。

密码算法（cipher）一般不是秘密，而是广为人知、经过深入研究的标准。所以，密码算法（cipher）保护数据的能力完全取决于密钥的保密。如果攻击者能得知、推断或猜出密钥，一切保护就都没了。因此，如果依靠密码学来保护，就必须在密钥的选择和保密上极其小心。一条好的原则是：把密钥存放在尽可能少的地方，存放尽可能短的时间，让尽可能少的人能拿到。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

参考文献

[D88] "The First Ten Years of Public Key Cryptography" by Whitfield Diffie. Communications of the ACM, Vol. 76, No. 5, May 1988. *A description of the complex history of where public key cryptography came from.*

[D+14] "The Matter of Heartbleed" by Zakir Durumeric, James Kasten, David Adrian, J. Alex Halderman, Michael Bailey, Frank Li, Nicholas Weaver, Johanna Amann, Jethro Beekman, Mathias Payer, and Vern Paxson. Proceedings of the 2014 Conference on Internet Measurement Conference. *A good description of the Heartbleed vulnerability in OpenSSL and its impact on the Internet as a whole. Worth reading for the latter, especially, as it points out how one small bug in one critical piece of system software can have a tremendous impact.*

[G02] "Lessons Learned in Implementing and Deploying Crypto Software" by Peter Gutmann. Usenix Security Symposium, 2002. *A good analysis of the many ways in which poor use of a perfectly good cipher can totally compromise your software, backed up by actual cases of the problems occurring in the real world.*

[G17] "SHA-1 Shattered" by Google. https://shattered.io, 2017. *A web site describing details of how Google demonstrated the insecurity of the SHA-1 cryptographic hashing function. The web site provides general details, but also includes a link to a technical paper describing exactly how it was done.*

[GW96] "Randomness and the Netscape Browser" by Ian Goldberg and David Wagner. Dr. Dobbs Journal, January 1996. *Another example of being able to deduce keys that were not properly created and handled, in this case by guessing the inputs to the random number generator used to create the keys. Aren't attackers clever? Don't you wish they weren't?*

[K96] "The Codebreakers" by David Kahn. Scribner Publishing, 1996. *A long, but readable, history of cryptography, its uses, and how it is attacked.*

[S96] "Applied Cryptography" by Bruce Schneier. Jon Wiley and Sons, Inc., 1996. *A detailed description of how to use cryptography in many different circumstances, including example source code.*

[V16] "House of Keys: 9 Months later… 40% Worse" by Stefan Viehbock. Available on: blog.sec-consult.com/2016/09/house-of-keys-9-months-later-40-worse.html. *A web page describing the unfortunate ubiquity of the same private key being used in many different embedded devices.*

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)